

6.3 Double-Ended Queues

We next consider a queue-like data structure that supports insertion and deletion at both the front and the back of the queue. Such a structure is called a *double-ended queue*, or *deque*, which is usually pronounced “deck” to avoid confusion with the dequeue method of the regular queue ADT, which is pronounced like the abbreviation “D.Q.”

The deque abstract data type is more general than both the stack and the queue ADTs. The extra generality can be useful in some applications. For example, we described a restaurant using a queue to maintain a waitlist. Occasionally, the first person might be removed from the queue only to find that a table was not available; typically, the restaurant will reinsert the person at the *first* position in the queue. It may also be that a customer at the end of the queue may grow impatient and leave the restaurant. (We will need an even more general data structure if we want to model customers leaving the queue from other positions.)

6.3.1 The Deque Abstract Data Type

The deque abstract data type is richer than both the stack and the queue ADTs. To provide a symmetrical abstraction, the deque ADT is defined to support the following update methods:

- `addFirst(e)`: Insert a new element *e* at the front of the deque.
- `addLast(e)`: Insert a new element *e* at the back of the deque.
- `removeFirst()`: Remove and return the first element of the deque (or null if the deque is empty).
- `removeLast()`: Remove and return the last element of the deque (or null if the deque is empty).

Additionally, the deque ADT will include the following accessors:

- `first()`: Returns the first element of the deque, without removing it (or null if the deque is empty).
- `last()`: Returns the last element of the deque, without removing it (or null if the deque is empty).
- `size()`: Returns the number of elements in the deque.
- `isEmpty()`: Returns a boolean indicating whether the deque is empty.

We formalize the deque ADT with the Java interface shown in Code Fragment 6.14.

```

1  /**
2   * Interface for a double-ended queue: a collection of elements that can be inserted
3   * and removed at both ends; this interface is a simplified version of java.util.Deque.
4   */
5  public interface Deque<E> {
6      /** Returns the number of elements in the deque. */
7      int size();
8      /** Tests whether the deque is empty. */
9      boolean isEmpty();
10     /** Returns, but does not remove, the first element of the deque (null if empty). */
11     E first();
12     /** Returns, but does not remove, the last element of the deque (null if empty). */
13     E last();
14     /** Inserts an element at the front of the deque. */
15     void addFirst(E e);
16     /** Inserts an element at the back of the deque. */
17     void addLast(E e);
18     /** Removes and returns the first element of the deque (null if empty). */
19     E removeFirst();
20     /** Removes and returns the last element of the deque (null if empty). */
21     E removeLast();
22 }

```

Code Fragment 6.14: A Java interface, Deque, describing the double-ended queue ADT. Note the use of the generic parameterized type, E, allowing a deque to contain elements of any specified class.

Example 6.5: The following table shows a series of operations and their effects on an initially empty deque *D* of integers.

Method	Return Value	<i>D</i>
addLast(5)	—	(5)
addFirst(3)	—	(3, 5)
addFirst(7)	—	(7, 3, 5)
first()	7	(7, 3, 5)
removeLast()	5	(7, 3)
size()	2	(7, 3)
removeLast()	3	(7)
removeFirst()	7	()
addFirst(6)	—	(6)
last()	6	(6)
addFirst(8)	—	(8, 6)
isEmpty()	false	(8, 6)
last()	6	(8, 6)

6.3.2 Implementing a Deque

We can implement the deque ADT efficiently using either an array or a linked list for storing elements.

Implementing a Deque with a Circular Array

If using an array, we recommend a representation similar to the `ArrayQueue` class, treating the array in circular fashion and storing the index of the first element and the current size of the deque as fields; the index of the last element can be calculated, as needed, using modular arithmetic.

One extra concern is avoiding use of negative values with the modulo operator. When removing the first element, the front index is advanced in circular fashion, with the assignment $f = (f+1) \% N$. But when an element is inserted at the front, the first index must effectively be decremented in circular fashion and it is a mistake to assign $f = (f-1) \% N$. The problem is that when f is 0, the goal should be to “decrement” it to the other end of the array, and thus to index $N-1$. However, a calculation such as $-1 \% 10$ in Java results in the value -1 . A standard way to decrement an index circularly is instead to assign $f = (f-1+N) \% N$. Adding the additional term of N before the modulus is calculated assures that the result is a positive value. We leave details of this approach to Exercise P-6.40.

Implementing a Deque with a Doubly Linked List

Because the deque requires insertion and removal at both ends, a doubly linked list is most appropriate for implementing all operations efficiently. In fact, the `DoublyLinkedList` class from Section 3.4.1 already implements the entire `Deque` interface; we simply need to add the declaration “**implements** `Deque<E>`” to that class definition in order to use it as a deque.

Performance of the Deque Operations

Table 6.5 shows the running times of methods for a deque implemented with a doubly linked list. Note that every method runs in $O(1)$ time.

Method	Running Time
size, isEmpty	$O(1)$
first, last	$O(1)$
addFirst, addLast	$O(1)$
removeFirst, removeLast	$O(1)$

Table 6.5: Performance of a deque realized by either a circular array or a doubly linked list. The space usage for the array-based implementation is $O(N)$, where N is the size of the array, while the space usage of the doubly linked list is $O(n)$ where $n < N$ is the actual number of elements in the deque.

6.3.3 Deques in the Java Collections Framework

The Java Collections Framework includes its own definition of a deque, as the `java.util.Deque` interface, as well as several implementations of the interface including one based on use of a circular array (`java.util.ArrayDeque`) and one based on use of a doubly linked list (`java.util.LinkedList`). So, if we need to use a deque and would rather not implement one from scratch, we can simply use one of those built-in classes.

As is the case with the `java.util.Queue` class (see page 240), the `java.util.Deque` provides duplicative methods that use different techniques to signal exceptional cases. A summary of those methods is given in Table 6.6.

Our Deque ADT	Interface <code>java.util.Deque</code>	
	throws exceptions	returns special value
<code>first()</code>	<code>getFirst()</code>	<code>peekFirst()</code>
<code>last()</code>	<code>getLast()</code>	<code>peekLast()</code>
<code>addFirst(<i>e</i>)</code>	<code>addFirst(<i>e</i>)</code>	<code>offerFirst(<i>e</i>)</code>
<code>addLast(<i>e</i>)</code>	<code>addLast(<i>e</i>)</code>	<code>offerLast(<i>e</i>)</code>
<code>removeFirst()</code>	<code>removeFirst()</code>	<code>pollFirst()</code>
<code>removeLast()</code>	<code>removeLast()</code>	<code>pollLast()</code>
<code>size()</code>	<code>size()</code>	
<code>isEmpty()</code>	<code>isEmpty()</code>	

Table 6.6: Methods of our deque ADT and the corresponding methods of the `java.util.Deque` interface.

When attempting to access or remove the first or last element of an *empty* deque, the methods in the middle column of Table 6.6—that is, `getFirst()`, `getLast()`, `removeFirst()`, and `removeLast()`—throw a `NoSuchElementException`. The methods in the rightmost column—that is, `peekFirst()`, `peekLast()`, `pollFirst()`, and `pollLast()`—simply return the null reference when a deque is empty. In similar manner, when attempting to add an element to an end of a deque with a capacity limit, the `addFirst` and `addLast` methods throw an exception, while the `offerFirst` and `offerLast` methods return false.

The methods that handle bad situations more gracefully (i.e., without throwing exceptions) are useful in applications, known as producer-consumer scenarios, in which it is common for one component of software to look for an element that may have been placed in a queue by another program, or in which it is common to try to insert an item into a fixed-sized buffer that might be full. However, having methods return null when empty are not appropriate for applications in which null might serve as an actual element of a queue.